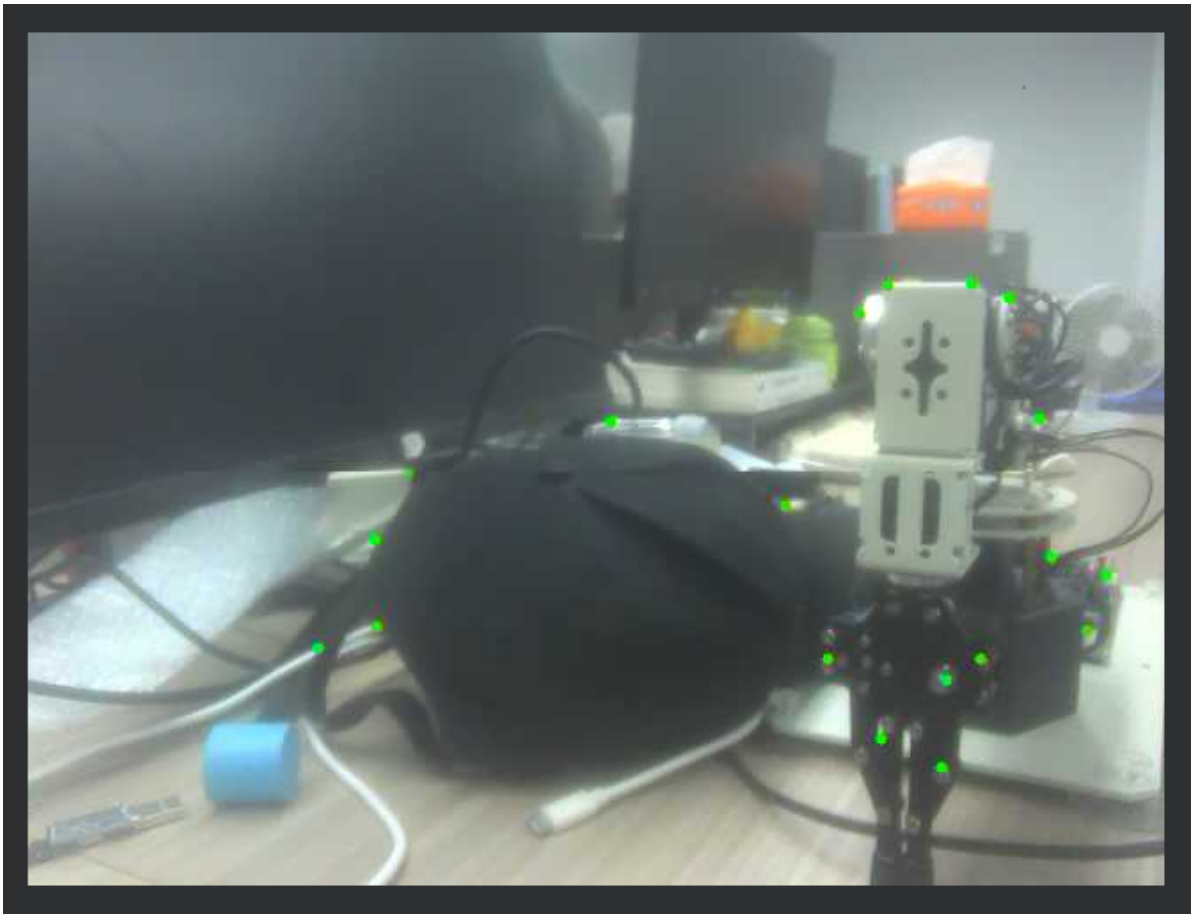
彩色图角点检测

彩色图角点检测
- 例程效果
- 原理讲解
  - 什么是图像RGB888角点检测
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置帧率计时器
    - 设置角点检测参数
    - 图像处理与角点检测
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/30.rgb888_find_corners.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像角点检测，检测并显示图像中的关键角点，用于特征提取和物体跟踪。



原理讲解

什么是图像RGB888角点检测

图像RGB888角点检测是一种计算机视觉技术，用于识别彩色图像（RGB888格式）中亮度或颜色变化剧烈的点（即角点），这些点通常位于物体的边缘或角落，具有较高的特征性。角点检测基于彩色图像的灰度转换或直接处理，常用算法如 Shi-Tomasi 角点检测器，能高效提取稳定的关键点。

- **操作原理**：角点检测算法首先将 RGB888 图像转换为灰度图像或直接在彩色空间计算梯度矩阵（Hessian 矩阵），然后评估每个像素的角点响应值（基于局部窗口的特征值）。通过设置质量阈值和最小距离，筛选出高质量的角点。公式涉及计算像素的特征值 λ1 和 λ2，如果 min(λ1, λ2) > 阈值，则认为是角点。该过程在 RGB888 图像上进行，以利用彩色信息增强检测精度。
- **效果**：检测结果突出图像中的关键点，并在图像上标记它们，提高视觉分析的准确性。处理后，角点可用于后续任务如物体跟踪或匹配，但检测精度受图像噪声、颜色失真和参数影响。
- **应用场景**：RGB888角点检测广泛用于实时物体跟踪（如视觉SLAM）、运动检测、图像配准和机器人视觉等领域。在嵌入式设备上，由于其计算效率高，它适合资源受限的环境，如摄像头实时处理彩色图像。

在实际应用中，通过调整参数，可以优化检测的鲁棒性和数量，以适应不同的图像场景和颜色复杂性。

代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *      # 摄像头接口 / Camera interface
from media.display import *     # 显示接口 / Display interface
from media.media import *       # 媒体资源管理器 / Media manager
import _thread
import cv_lite                  # cv_lite扩展模块 / cv_lite extension module
import ulab.numpy as np
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960,fps=60)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 60 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合进行角点检测以利用颜色信息）。

初始化显示模块

```
Display.init(Display.VIRT, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=50)
```

初始化显示模块，使用虚拟显示驱动（Display.VIRT），分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置帧率计时器

```
clock = time.clock()  # 启动帧率计时器 / Start FPS timer
```

初始化帧率计时器，用于计算和显示每秒处理的帧数（FPS）。

设置角点检测参数

```
max_corners        = 20         # 最大角点数 / Maximum number of corners
quality_level      = 0.01       # Shi-Tomasi质量因子 / Corner quality factor (0.01 ~
0.1)
min_distance       = 20.0       # 最小角点距离 / Minimum distance between corners
```

- `max_corners`：最大检测角点数量，限制输出结果。
- `quality_level`：角点质量阈值，用于筛选高响应值的角点。
- `min_distance`：角点之间最小距离阈值，避免检测到过于密集的点。

图像处理与角点检测

```
while True:
    clock.tick()

    # 拍摄当前帧图像 / Capture current frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference

    # 调用角点检测函数，返回角点数组 [x0, y0, x1, y1, ...]
    corners = cv_lite.rgb888_find_corners(
        image_shape, img_np,
        max_corners,
        quality_level,
        min_distance
    )

    # 遍历角点数组，绘制角点 / Draw detected corners
    for i in range(0, len(corners), 2):
        x = corners[i]
        y = corners[i + 1]
        img.draw_circle(x, y, 3, color=(0, 255, 0), fill=True)

    # 显示图像 / Display image with corners
    Display.show_image(img)

    # 进行垃圾回收 / Perform garbage collection
```

```
    gc.collect()

    # 打印当前帧率和角点数量 / Print current FPS and number of corners
    print("fps:", clock.fps(), "| corners:", len(corners) // 2)
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **角点检测处理**：调用 `cv_lite.rgb888_find_corners()` 函数执行角点检测，返回角点坐标数组。
- **绘制和显示**：遍历检测到的角点，在图像上绘制绿色圆圈标记，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率和角点数量。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `max_corners`：最大角点数，值越大（如 10-50）检测更多角点，但可能增加计算负担；值越小（如 5）减少点数以提高速度，建议根据图像复杂度从 20 开始测试。
- `quality_level`：Shi-Tomasi 质量因子，值越高（如 0.01-0.1）要求角点响应更强，检测更精确但数量可能减少；值过低（如 0.001）可能检测到噪声，建议从 0.01 开始微调。
- `min_distance`：最小角点距离，值越大（如 10-50）确保角点间距，避免密集点；值越小（如 5）检测更密集但可能包含冗余，建议根据图像分辨率和目标场景调整，从 20 开始测试。